

Índice

1. Introduccion	3
2. Alcance y casos de uso	3
3. Análisis del conjunto de datos	4
3.1. Estructura del dataset	4
3.2. Variantes disponibles	4
3.3. Conversión de divisas	5
3.4. Campos utilizados por consulta	5
4. Procesamiento y flujo de datos	5
4.1. Cadenas de procesamiento por consulta	6
4.1.1. Consulta 1 — Transacciones USD menores a 50	6
4.1.2. Consulta 2 — Transacción USD máxima por banco	6
4.1.3. Consulta 3 — Anomalías por comparación de promedios	7
4.1.4. Consulta 4 — Detección del patrón scatter-gather	7
4.1.5. Consulta 5 — Conteo de transacciones Wire/ACH bajo 1 USD	7
4.2. Flujo de mensajes	8
4.2.1. Consulta 1	8
4.2.2. Consulta 2	9
4.2.3. Consulta 3	9
4.2.4. Consulta 4	10
4.2.5. Consulta 5	11
4.3. Actividades de los procesos en el sistema	11
4.3.1. Pre-procesamiento - Gateway	12
4.3.2. Consulta 3	13
4.3.3. Consulta 4	14
4.3.4. Post-procesamiento — Cierre del cliente	15
4.4. Arquitectura e Infraestructura	15
4.5. Vista de Desarrollo	16
4.5.1. Diagrama de Paquetes	16
4.6. Vista Lógica	17
4.6.1. Diagrama de Clases	17
4.7. Vista Fisica	18
5. Tareas a implementar	19
5.1. División entre integrantes	19

1. Introduccion

En el presente trabajo se desea desarrollar un sistema distribuido para el analisis y deteccion de anomalias en transacciones entre cuentas bancarias. Para ello se provee un conjunto de consultas predefinidas que ayuden al correspondiente analisis.

2. Alcance y casos de uso

El proyecto *Money Laundering Analysis* consiste en un sistema distribuido que analiza el extracto de transacciones realizadas entre cuentas bancarias en busca de anomalías asociadas a patrones de lavado de activos. El sistema recibe como entrada un conjunto de datos de transacciones bancarias y produce, para cada consulta definida, un reporte con los resultados correspondientes.

El sistema soporta múltiples clientes concurrentes. Cada cliente envía su propio conjunto de transacciones y recibe de forma independiente los resultados de las cinco consultas. Las ejecuciones de distintos clientes son aisladas entre sí: un cliente no tiene acceso a los datos ni a los resultados de otro. No se impone ninguna restricción sobre el subconjunto de registros que cada cliente elija enviar.

Las consultas que el sistema debe responder para cada cliente son las siguientes:

Las consultas que el sistema debe responder son las siguientes:

- **Consulta 1:** Obtener cuenta de origen, cuenta de destino y monto de transacciones en USD con monto menor a 50.
- **Consulta 2:** Obtener nombre de banco, cuenta de origen y monto de la transacción USD máxima por banco.
- **Consulta 3:** Obtener cuenta de origen y monto de transacciones USD en el período [2022-09-06, 2022-09-15] cuyo monto sea menor a un centésimo del promedio del mismo formato de pago en el período [2022-09-01, 2022-09-05].
- **Consulta 4:** Detectar cuentas que presenten el patrón *scatter-gather* con una única cuenta de separación, considerando cuentas que hayan realizado transferencias USD hacia al menos 5 cuentas distintas en el período [2022-09-01, 2022-09-05].
- **Consulta 5:** Contar la cantidad de transacciones del período [2022-09-01, 2022-09-05] con formato de pago Wire o ACH cuyo monto convertido a USD sea menor a 1.

A continuación se presenta el diagrama de casos de uso del sistema, donde se ilustran las interacciones entre el cliente y cada una de las consultas disponibles.

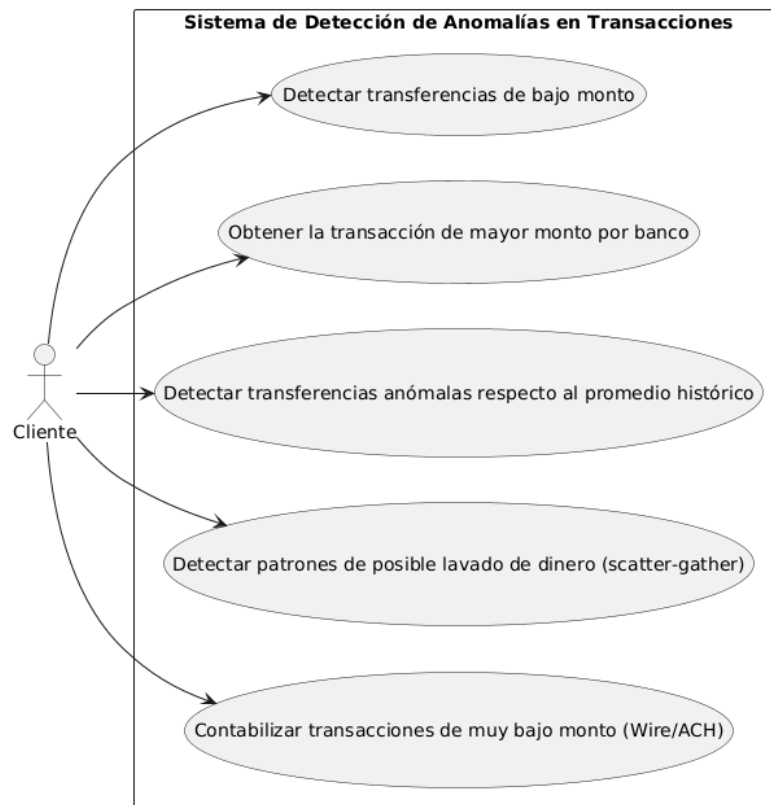


Figura 1: Diagrama de casos de uso, mostrando las interacciones entre el cliente y el sistema.

3. Análisis del conjunto de datos

El sistema trabaja sobre el dataset sintético de transacciones bancarias publicado por IBM para detección de lavado de activos, disponible en Kaggle¹. Este dataset fue generado mediante un simulador que modela el ciclo completo del lavado de activos: *placement*, *layering* e *integration*, y cada transacción se encuentra etiquetada como legítima o sospechosa.

3.1. Estructura del dataset

El dataset se provee en formato CSV. Cada fila representa una transacción entre dos cuentas bancarias y contiene las siguientes columnas:

3.2. Variantes disponibles

El dataset se distribuye en seis variantes combinando dos grupos (*HI* con mayor tasa de lavado y *LI* con menor tasa) y tres tamaños. La siguiente tabla resume sus características principales:

Dado que las consultas 3 y 4 requieren datos hasta el 15 de septiembre de 2022, las variantes *Small* no son suficientes. El sistema está diseñado para operar sobre las variantes *Medium* o *Large*, siendo *HI-Medium* y *LI-Medium* las utilizadas como referencia para validar los resultados contra el notebook patrón provisto por la cátedra.

¹<https://www.kaggle.com/datasets/ealtman2019/ibm-transactions-for-anti-money-laundering-aml>

Campo	Descripción
Timestamp	Fecha y hora de la transacción
From Bank	Banco de origen
Account (origen)	Cuenta de origen
To Bank	Banco de destino
Account (destino)	Cuenta de destino
Amount Paid	Monto enviado
Payment Currency	Moneda de la transacción
Amount Received	Monto recibido
Receiving Currency	Moneda recibida
Payment Format	Formato de pago (Wire, ACH, etc.)
Is Laundering	Etiqueta: 0 legítima, 1 sospechosa

Cuadro 1: Columnas del dataset de transacciones

3.3. Conversión de divisas

El dataset contiene transacciones en múltiples monedas. Las consultas 1, 2, 3 y 4 operan sobre montos USD, en cuanto a la 5 se hace sobre montos convertidos a USD, por lo que es necesario convertir cada transacción a esa moneda de referencia utilizando la tasa de cambio correspondiente a la fecha de la transacción.

Para esto, el sistema consulta la API pública de Frankfurter (<https://api.frankfurter.app>) durante la fase de inicialización, obteniendo las tasas de cambio diarias para el período cubierto por el dataset. Estas tasas se persisten localmente antes de iniciar el procesamiento, eliminando la dependencia externa del camino crítico del pipeline. Se tratan como datos de referencia inmutables durante toda la ejecución, dado que corresponden a fechas históricas de 2022.

3.4. Campos utilizados por consulta

No todos los campos del dataset son relevantes para todas las consultas. La siguiente tabla detalla qué campos consume cada una:

Consulta	Campos utilizados
Q1	Account (origen), Account (destino), Amount Paid, Payment Currency
Q2	From Bank, Account (origen), Amount Paid, Payment Currency
Q3	Account (origen), Amount Paid, Payment Currency, Payment Format, Timestamp
Q4	Account (origen), Account (destino), Amount Paid, Payment Currency, Timestamp
Q5	Amount Paid, Payment Currency, Payment Format, Timestamp

Cuadro 2: Campos del dataset utilizados por cada consulta

4. Procesamiento y flujo de datos

En esta sección se describe cómo la información atraviesa el sistema desde su ingesta hasta la producción del reporte final. Se presentan primero las cadenas de procesamiento individuales de cada consulta como grafos acíclicos dirigidos (DAG), para luego mostrar el flujo completo del sistema integrando todas las consultas en paralelo.

4.1. Cadenas de procesamiento por consulta

Antes de ramificarse en los pipelines individuales, todas las transacciones atraviesan un filtro común que descarta aquellas cuya moneda no sea USD. Este filtro inicial reduce el volumen de datos de forma temprana, beneficiando a todas las consultas que operan exclusivamente sobre transacciones en dólares. A partir de este punto, los datos fluyen en paralelo por cinco pipelines independientes, uno por consulta. A continuación se describe el DAG del sistema.

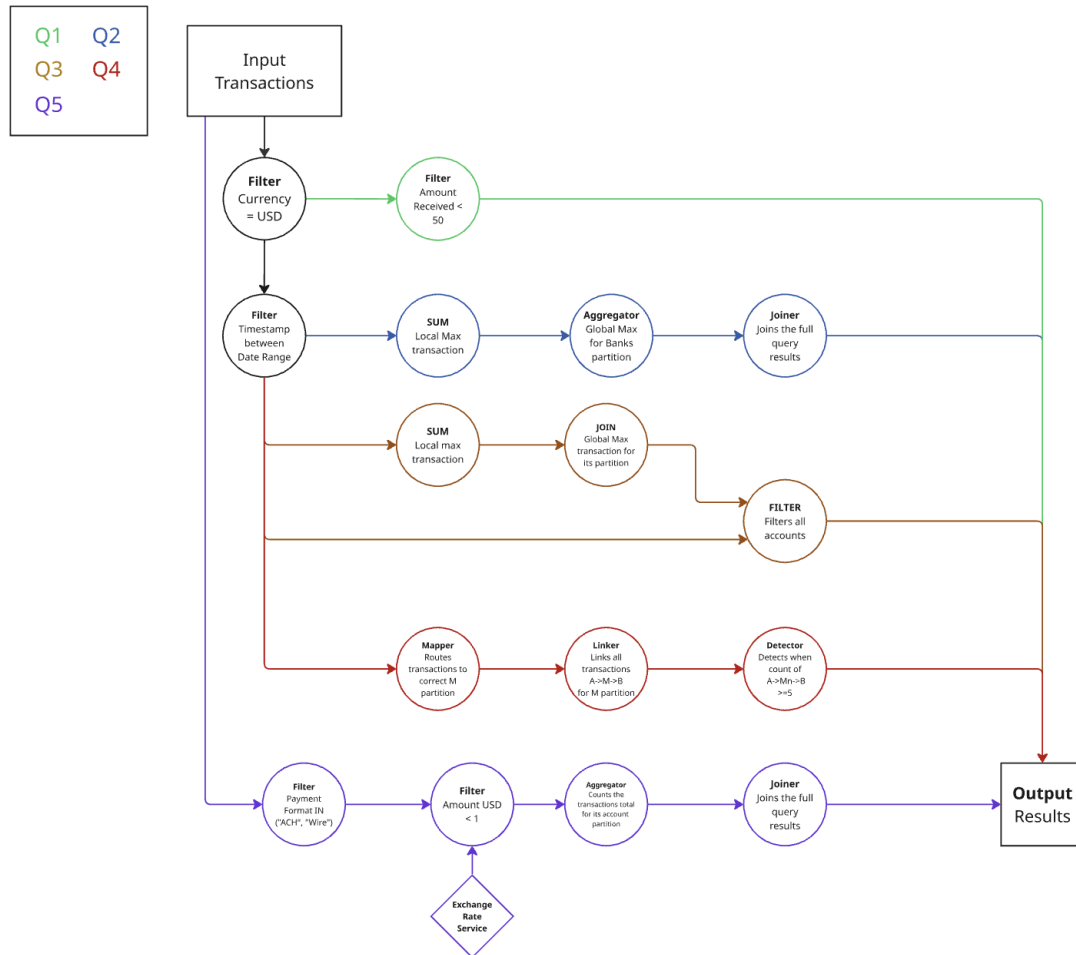


Figura 2: DAG general del sistema con los cinco pipelines en paralelo

4.1.1. Consulta 1 — Transacciones USD menores a 50

El pipeline de la Consulta 1 es el más directo del sistema. Luego del filtro común de moneda, un único nodo *Filter* verifica que el monto recibido sea menor a 50 dólares. Las transacciones que superan esta condición son enviadas directamente al resultado de salida como tuplas (cuenta origen, cuenta destino, monto). No se requiere ninguna etapa de agregación ya que el resultado es un conjunto de registros individuales, no una reducción.

4.1.2. Consulta 2 — Transacción USD máxima por banco

La Consulta 2 incorpora un segundo filtro previo que retiene únicamente las transacciones cuyo *Timestamp* se encuentre dentro del rango de fechas relevante. Las transacciones restantes son

procesadas por el nodo *SUM*, que calcula el máximo local de monto por banco para su partición. El nodo *Aggregator* consolida los máximos parciales de todas las réplicas y determina el máximo global por banco. Finalmente, el nodo *Joiner* enriquece cada resultado con el nombre completo del banco y la cuenta de origen asociada, produciendo el reporte final.

Cabe aclarar que el cliente envía junto con las transferencias un archivo de cuentas que contiene el mapeo entre ID de banco y nombre. El Gateway rutea este archivo directamente al Joiner de Q2 que lo mantiene en memoria y lo aplica al enriquecer los resultados con el nombre completo del banco antes de devolver al Gateway. Para más detalle ver diagrama de secuencia.

4.1.3. Consulta 3 — Anomalías por comparación de promedios

La Consulta 3 comparte con Q2 el filtro de rango temporal, pero su pipeline se bifurca internamente en dos ramas que deben coordinarse antes de producir un resultado.

Rama base: el nodo *SUM* acumula la suma de montos y el conteo de transacciones agrupados por formato de pago, correspondientes al período [2022-09-01, 2022-09-05]. Al recibir la señal de fin de datos, el nodo *JOIN* consolida los aportes de todas las réplicas y calcula el promedio real por formato junto con el umbral correspondiente (promedio $\times$ 0.01). Estos umbrales son publicados como mensaje de control hacia la segunda rama.

Rama actual: el nodo *FILTER* acumula en un buffer las transacciones del período [2022-09-06, 2022-09-15] mientras espera el mensaje de control con los umbrales calculados por la rama base. Solo cuando ambas condiciones se satisfacen —buffer completo y umbrales disponibles— el nodo drena el buffer y filtra las transacciones cuyo monto sea inferior al umbral de su formato de pago, emitiendo las anomalías detectadas al resultado de salida.

4.1.4. Consulta 4 — Detección del patrón scatter-gather

La Consulta 4 también parte del filtro de rango temporal y modela el problema como un análisis topológico sobre el grafo de transacciones, donde cada arista representa una transferencia entre dos cuentas.

El nodo *Mapper* recibe las transacciones filtradas y las enruta hacia la partición correcta según la cuenta intermedia (nodo *M*), garantizando que todas las aristas $A \rightarrow M$ y $M \rightarrow B$ de un mismo nodo intermedio sean procesadas por la misma réplica. El nodo *Linker* construye para cada cuenta intermedia el conjunto de relaciones $A \rightarrow M \rightarrow B$. Finalmente, el nodo *Detector* verifica la condición del patrón scatter-gather: que la cantidad de cuentas origen distintas que alimentan a la cuenta intermedia sea mayor o igual a 5. Las cuentas que cumplen esta condición son emitidas al resultado de salida.

4.1.5. Consulta 5 — Conteo de transacciones Wire/ACH bajo 1 USD

El pipeline de la Consulta 5 no parte del filtro de rango temporal compartido con Q2, Q3 y Q4, sino que aplica su propio conjunto de filtros en cascada siguiendo el principio de *predicate pushdown*: las operaciones más baratas se ejecutan primero para reducir el volumen antes de las costosas.

El primer nodo *Filter* retiene únicamente las transacciones del período [2022-09-01, 2022-09-05] con formato de pago Wire o ACH. El segundo nodo *Filter* convierte el monto a USD utilizando las tasas de cambio provistas por el *Exchange Rate Service* —obtenidas durante la fase de inicialización y distribuidas a todos los workers al arrancar— y descarta las transacciones cuyo monto convertido sea mayor o igual a 1. Las transacciones restantes son contadas por el nodo *Aggregator*, que acumula un conteo parcial local. El nodo *Joiner* consolida los conteos de todas las réplicas y produce el resultado final: un único entero con el total de transacciones que cumplen las condiciones.

4.2. Flujo de mensajes

En esta sección se detalla la comunicación entre los componentes del sistema para cada consulta, mostrando el intercambio de mensajes desde que el cliente envía las transacciones hasta que recibe el resultado. Todos los componentes intermedios se comunican exclusivamente a través del middleware de mensajería. El *Controller* interviene al final de cada etapa para coordinar la propagación de la señal de fin de transmisión (*EOF*) hacia el siguiente stage, garantizando que ningún componente emita su resultado parcial antes de haber procesado la totalidad de los mensajes que le corresponden.

4.2.1. Consulta 1

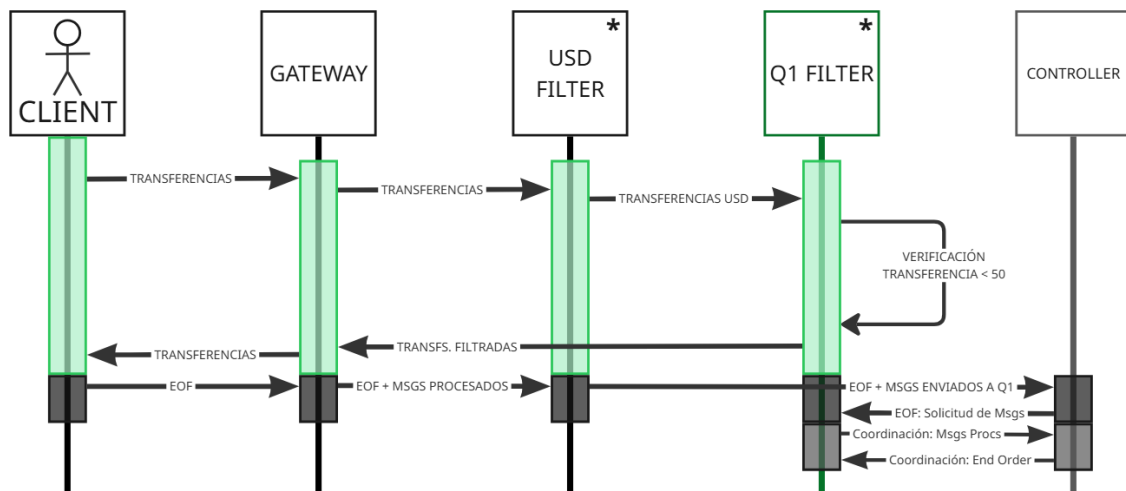


Figura 3: Diagrama de secuencia — Consulta 1

El cliente envía las transacciones al Gateway, que las reenvía al *USD Filter*. Este componente descarta las transacciones en moneda distinta a USD y pasa el resto al *Q1 Filter*, que aplica el criterio de monto menor a 50 y devuelve las transacciones que lo cumplen directamente al Gateway para su entrega al cliente. Al finalizar el envío de datos, el cliente emite un *EOF* que se propaga en cascada: el Gateway notifica al *Controller* cuántos mensajes procesó, el *USD Filter* informa cuántos pasaron el filtro de moneda, y el *Q1 Filter* hace lo propio antes de emitir su propio *EOF* hacia el *Controller*, que confirma la finalización de la consulta.

4.2.2. Consulta 2

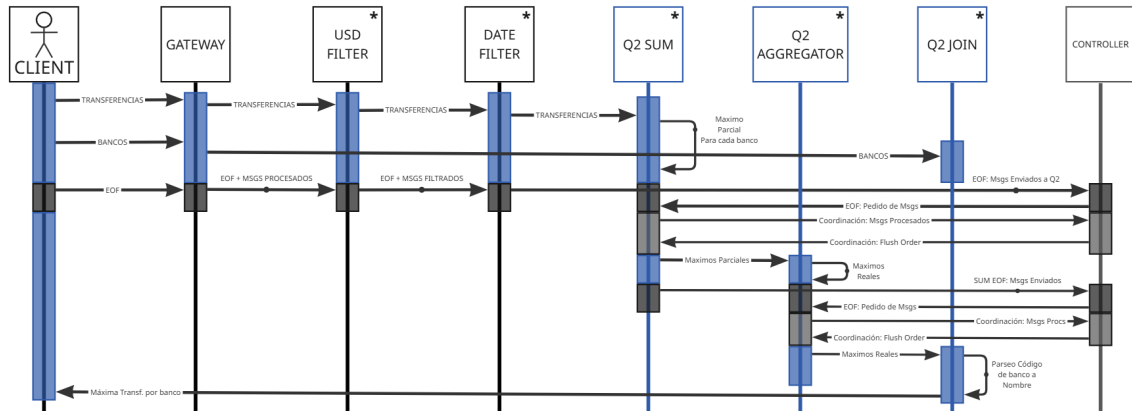


Figura 4: Diagrama de secuencia — Consulta 2

Las transacciones atraviesan el *USD Filter* y el *Date Filter* antes de llegar al *Q2 SUM*, que calcula el máximo parcial por banco para su partición. Una vez recibida la señal de fin de datos, las réplicas del *Q2 SUM* se coordinan entre sí antes de enviar sus máximos parciales al *Q2 Aggregator*. Este último consolida los aportes de todas las réplicas, calcula los máximos reales por banco y los envía al *Q2 Join*, que produce el resultado final y lo devuelve al Gateway para su entrega al cliente.

4.2.3. Consulta 3

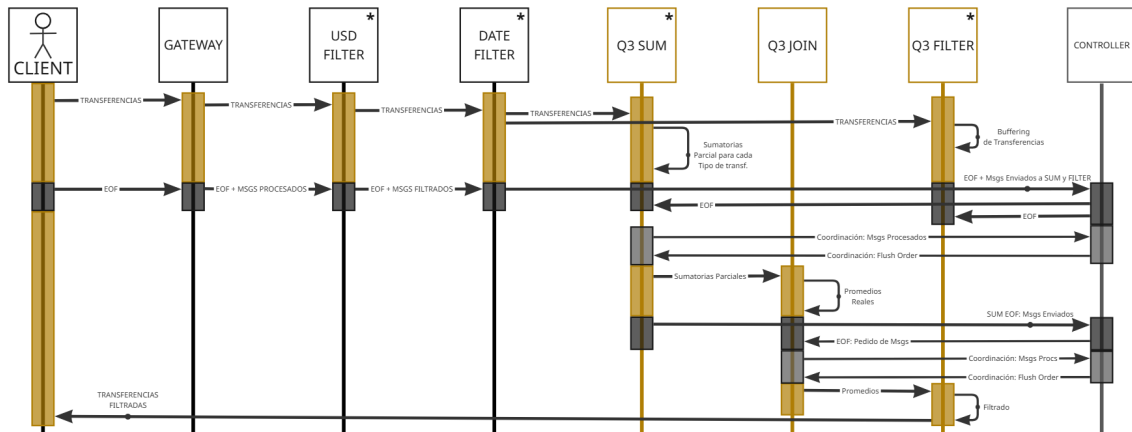


Figura 5: Diagrama de secuencia — Consulta 3

Las transacciones se "bifurcan" a *Q3 SUM* y *Q3 FILTER*. Esta bifurcación es definida ya que los promedios se definen con fechas distintas a las transacciones a filtrar. Las instancias de *Q3 SUM* se encargan de tomar la sumatoria parcial de monto y cantidad de transacciones que procesan por formato de pago. Al llegar el EOF, SUM se encarga de coordinar con controller para enviar los datos a *Q3 JOIN*, que crea los promedios reales por formato de pago.

Q3 FILTER Recibe los datos de transacciones hasta recibir el EOF del Controller, buffereando los datos hasta recibir los promedios provenientes del *Q3 JOIN*. Finalmente, las instancias de *Q3 FILTER* devuelven al cliente las transacciones que cumplan con la condición impuesta por el centésimo del promedio.

4.2.5. Consulta 5

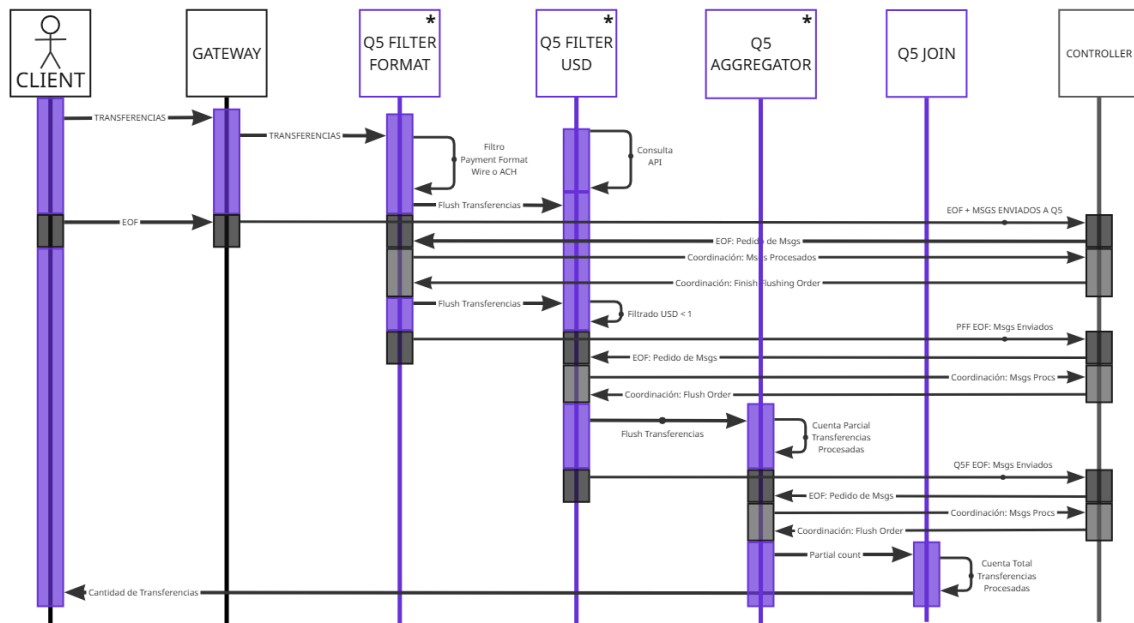


Figura 7: Diagrama de secuencia — Consulta 5

Las transacciones llegan al Q5 Filter Format, que aplica dos condiciones en una sola pasada: retiene solo las transacciones del periodo [2022-09-01, 2022-09-05] con formato de pago Wire o ACH, descartando el resto. Las que pasan son enviadas al Filter Q5 USD, que convierte el monto a USD utilizando las cotizaciones diarias cargadas desde rates.csv al momento de inicialización, aplicando la tasa correspondiente a la fecha de cada transacción y descarta aquellas cuyo monto convertido sea mayor o igual a 1. Las transacciones restantes son enviadas al Aggregator que incrementa su contador parcial en streaming sin esperar EOF.

A diferencia de Q3 donde el filter q3 bloquea esperando los promedios del join, en Q5 todo el procesamiento ocurre en streaming: el filter usd convierte y filtra cada transacción al recibirla, y el Aggregator cuenta sin acumular estado más allá de un contador.

4.3. Actividades de los procesos en el sistema

En la siguiente sección se describe las actividades realizadas por los actores del sistema, tomando tres etapas: el pre-procesamiento, cómo el Gateway gestiona la concurrencia entre clientes, el procesamiento donde muestran la coordinación interna de las queries más complejas (3 y 4), y el post-procesamiento que muestra cómo convergen los resultados y finaliza el ciclo de vida del cliente.

4.3.1. Pre-procesamiento - Gateway

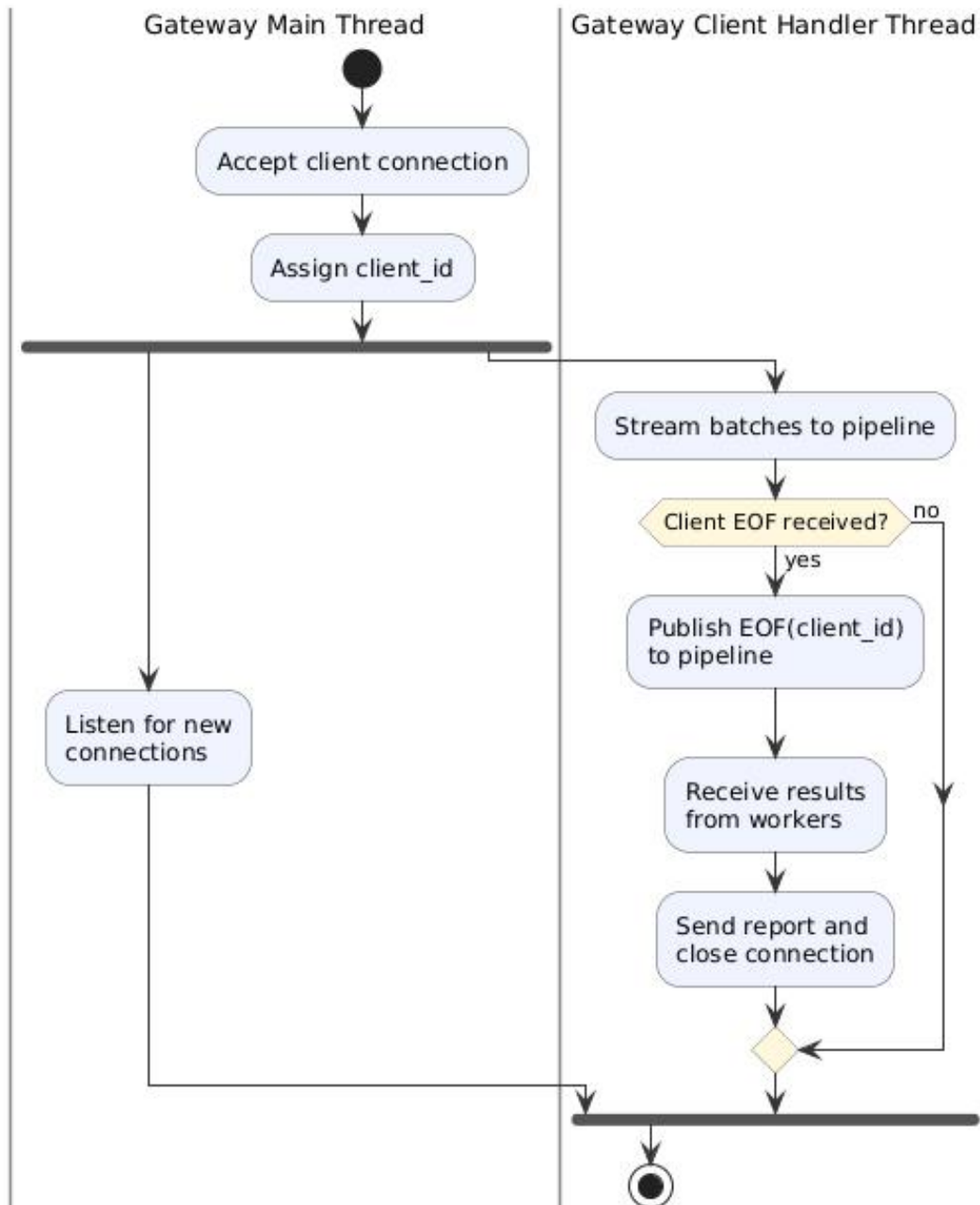


Figura 8: Diagrama de Actividades

El Gateway es el único punto de entrada al sistema y el responsable de gestionar la concurrencia entre clientes. Al aceptar una conexión, asigna un clientid único y spawnea un Client Handler Thread dedicado, volviendo inmediatamente a escuchar nuevas conexiones. Este fork es el mecanismo central de multi-cliente: el Main Thread nunca bloquea, permitiendo atender conexiones concurrentes de forma indefinida mientras cada thread maneja el ciclo de vida completo de su cliente de forma aislada.

El Client Handler streama los batches al pipeline etiquetados con el clientid, garantizando aislamiento de datos entre clientes que comparten los mismos workers. Al detectar el EOF del cliente, publica la signal de fin de datos al pipeline y notifica al Controller la cantidad de mensajes enviados por etapa, iniciando el proceso de coordinacion de cierre. A continuacion queda en espera de resultados.

4.3.2. Consulta 3

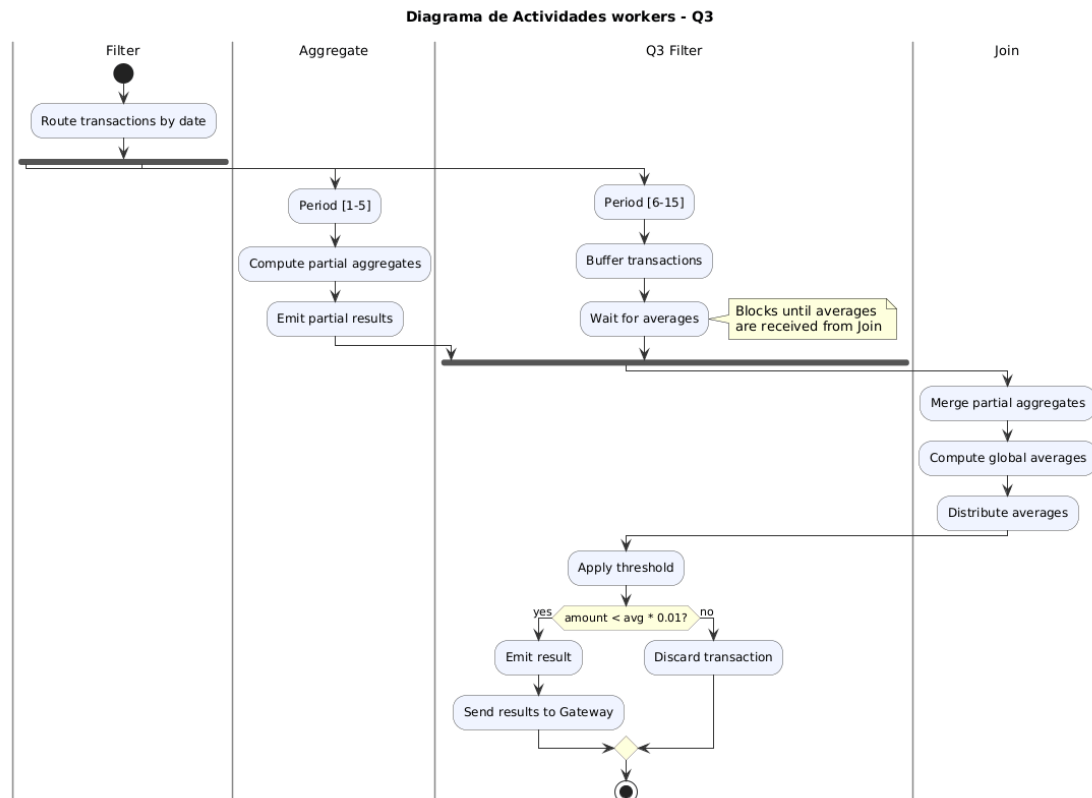


Figura 9: Diagrama de Actividades

La Consulta 3 fue elegida porque es la que tiene la coordinación más complicada: hay dos ramas que trabajan por separado pero que en algún momento sí o sí tienen que sincronizarse para poder dar un resultado.

Una decisión de diseño importante fue usar un fork, lo que implica que los cuatro workers arrancan totalmente independientes, sin un orquestador central que los esté coordinando. La sincronización se termina resolviendo de forma “natural” a través del middleware: el Q3 Filter queda esperando en el estado de “Wait for averages”, pero no porque alguien lo bloquee explícitamente, sino simplemente porque todavía no tiene mensajes para consumir hasta que el Join publique los promedios. Este es el único caso en todo el sistema donde un worker queda esperando a otro, y pasa justamente por la dependencia entre los dos períodos de la consulta.

En esa misma línea, se decidió bufferear en el Q3 Filter en lugar de procesar en streaming. La razón es se debe a que sin conocer el threshold, no hay forma de decidir qué transacciones se deberían emitir. Ese buffer crece de forma lineal con el volumen del período [6–15], y es el principal riesgo de memoria de esta consulta. Si el dataset fuera más grande que la RAM disponible, la alternativa sería volcar a disco con un merge externo.

Por último, la sincronización antes del Join refleja que ese nodo recién puede emitir resultados

cuando recibió todos los agregados parciales de las réplicas del SUM. Esta “barrera” asegura que el promedio calculado sea realmente global, y no un promedio parcial sobre solo una parte de los datos.

4.3.3. Consulta 4

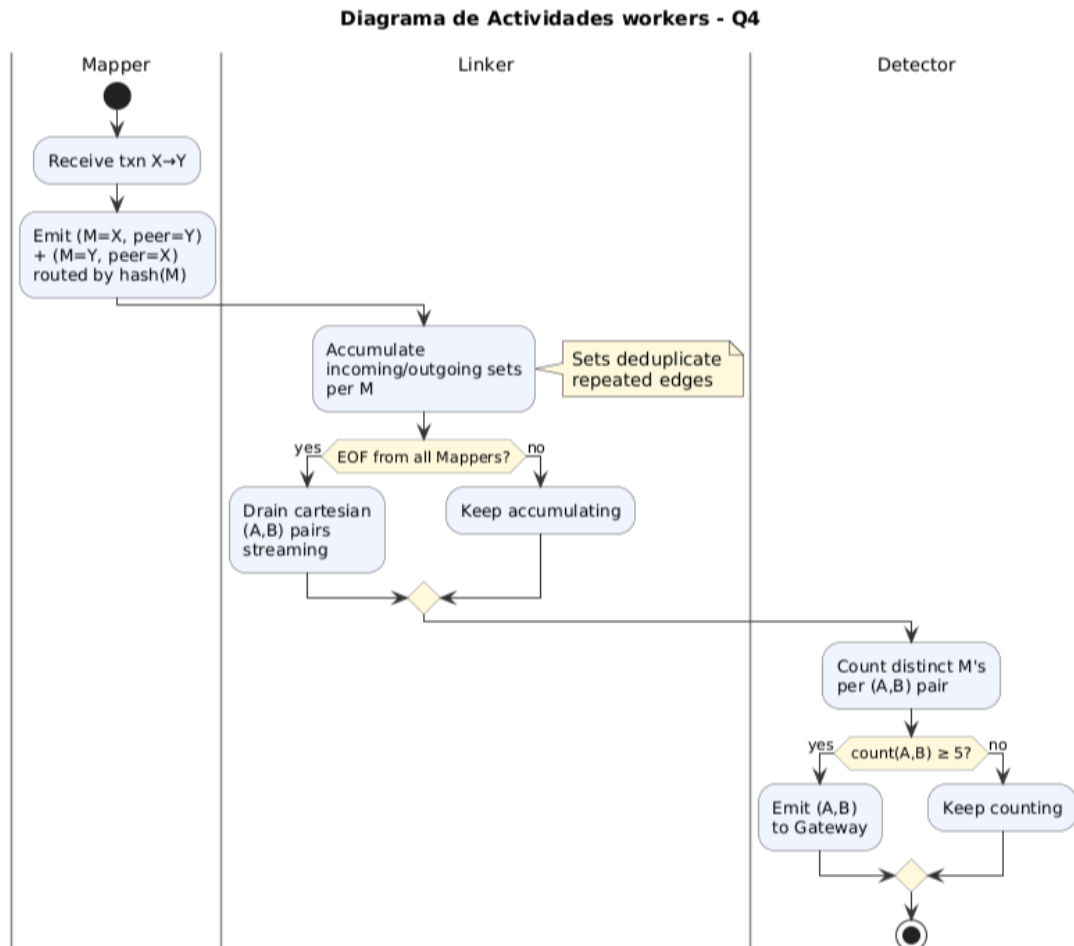


Figura 10: Diagrama de Actividades

La Consulta 4 presenta un patrón de coordinación distinto al de Q3: en lugar de dos ramas que convergen en un join, es una cadena de tres etapas donde cada una alimenta a la siguiente mediante un shuffle particionado.

El Mapper es la única etapa completamente stateless del sistema. Por cada transacción $X \rightarrow Y$ emite dos mensajes ruteados por $\text{hash}(M)$, garantizando que el Linker responsable de cada cuenta intermediaria reciba ambas patas del patrón ($A \rightarrow M$ y $M \rightarrow B$) sin coordinación entre réplicas.

El Linker acumula sets de cuentas por cada M de su partición. El uso de sets garantiza deduplicación natural de aristas repetidas. La barrera antes del drain refleja que el Linker no puede emitir hasta recibir EOF de todos los Mappers — garantía de haber visto la totalidad de las aristas de su partición. La emisión del producto cartesiano se hace en streaming, manteniendo memoria $O(1)$ durante el drain.

El Detector introduce el patrón de early-emit: a diferencia del Q3 Filter que bloquea esperando umbrales, el Detector emite resultados en streaming apenas detecta $\text{count}(A,B) \geq 5$. Una vez confirmado un par, se descarta del contador, acotando el estado al conjunto de pares aún no

confirmados.

4.3.4. Post-procesamiento — Cierre del cliente

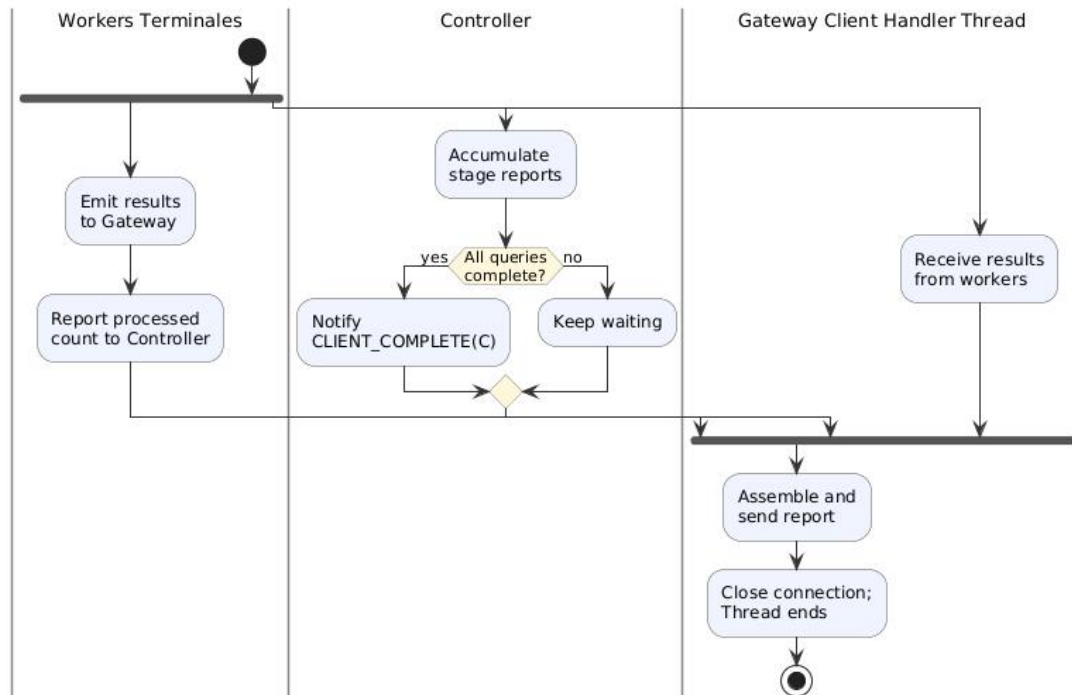


Figura 11: Diagrama de Actividades

Una vez que el pipeline procesó todos los datos, los workers terminales de cada query emiten sus resultados al Gateway en streaming y reportan al Controller la cantidad de mensajes procesados. El Controller actúa como barrera: acumula los reportes de todas las etapas y, cuando confirma que las cinco queries completaron su procesamiento, notifica `CLIENTCOMPLETE(C)` al Gateway. El join antes del ensamblado refleja que el Gateway necesita dos condiciones simultáneas para responder: haber recibido todos los resultados parciales de los workers y haber recibido la confirmación del Controller. Una vez satisfechas, el Client Handler Thread ensambla el reporte final, lo envía al cliente, cierra la conexión y finaliza — siendo el único proceso del sistema con un ciclo de vida acotado.

4.4. Arquitectura e Infraestructura

En la siguiente sección se muestra el diagrama de robustez, en el cual figura la interacción del client con nuestro sistema, y como dentro del mismo interactúan los diferentes actores, como escalan y flujo de datos entre ellos.

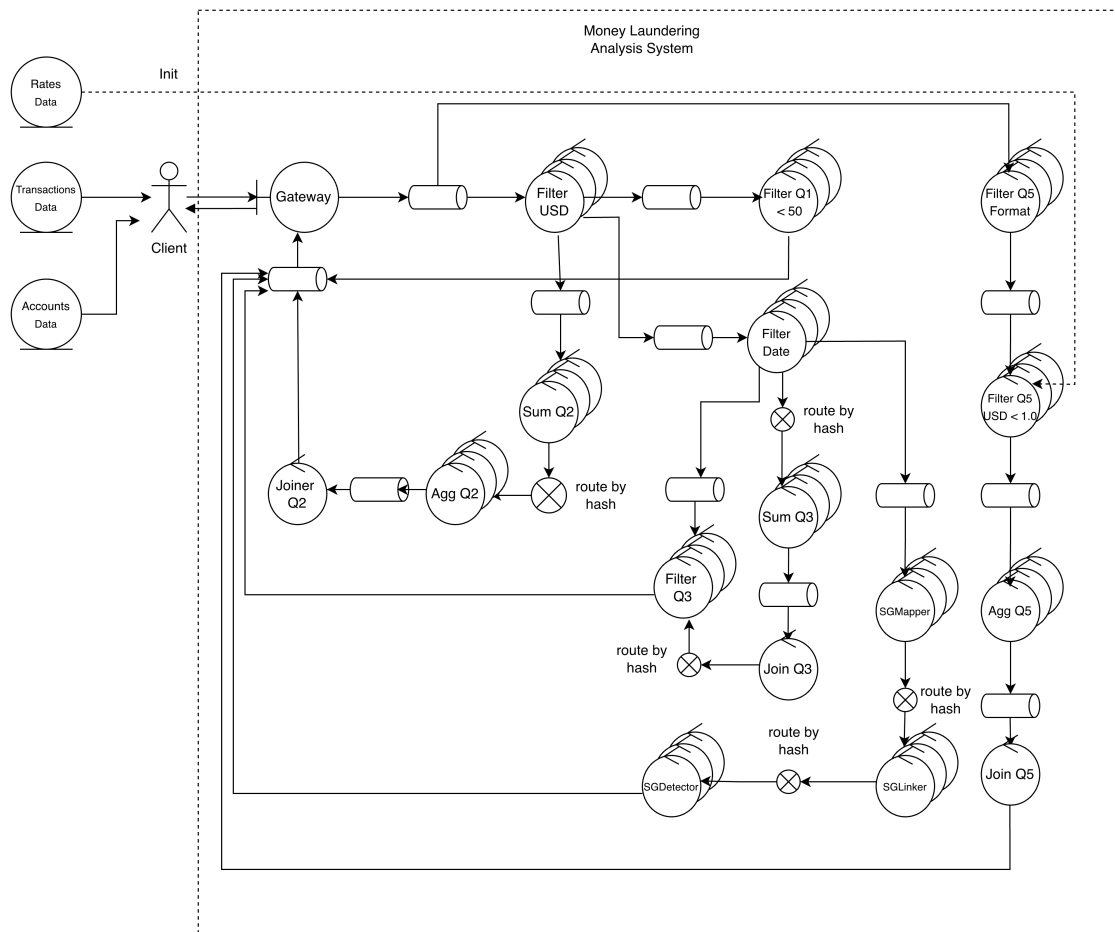


Figura 12: Diagrama de Robustez

En el diagrama se observa el flujo e interacción de las 5 queries, y como se hace uso de colas de RabbitMQ para la comunicación entre los diferentes workers/controladores. Un detalle a aclarar es la notación de exchange, que denota el ruteo por hash a los diferentes controladores del step siguiente en el pipeline. Esto está relacionado al nivel de escalamiento de nuestro sistema, el cual escala en dirección horizontal aumentando las replicas de los controladores en cada caso en particular.

4.5. Vista de Desarrollo

4.5.1. Diagrama de Paquetes

Los paquetes que se destacan:

- **Applications:** Contiene principalmente el cliente, que se comunica con el Gateway para enviar los datos de las transacciones a los workers correspondientes.
- **Messages:** Define los mensajes que se pueden utilizar para la comunicación entre Workers, y los datos enviados devuelta al cliente. Además contiene un módulo dedicado a serializar y deserializar grandes batches de mensajes.
- **Domain:** Define las estructuras internas de los datos.

- **Control:** Paquete Encargado de la comunicación y coordinación de la comunicación ante el evento de un End Of File (EOF). EOFController se encarga de comunicarse con las instancias de un mismo worker, verificar el correcto procesamiento de todos los datos enviados, y de enviar la flush order para liberar los datos al siguiente worker que corresponda.
- **Workers:** Son los encargados de procesar y transformar los datos correctamente según las consignas. Se divide en Filters, Sums, Joins, Aggregators y ScatterGather (Q4) según corresponda. Reciben datos del paquete de aplicación y se comunican con el paquete de control para coordinar durante el EOF.
- **External Services:** Se encarga de tener los datos correctos sobre la conversión de unidad de cambio. Se comunica con los workers correspondientes de Q5
- **RabbitMQ:** Encargado de definir las estructuras de comunicación entre los componentes. Todos los paquetes que necesiten comunicarse usan tanto exchanges o colas definidas en este paquete.

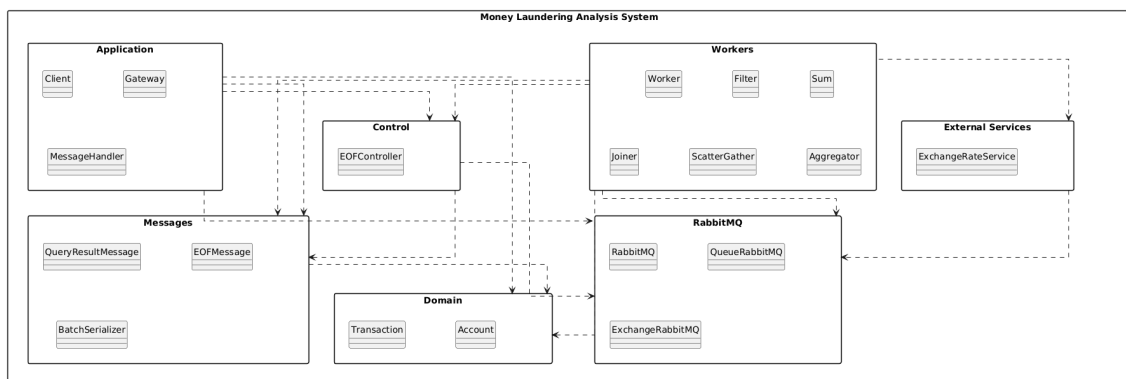


Figura 13: Diagrama de Paquetes

4.6. Vista Lógica

4.6.1. Diagrama de Clases

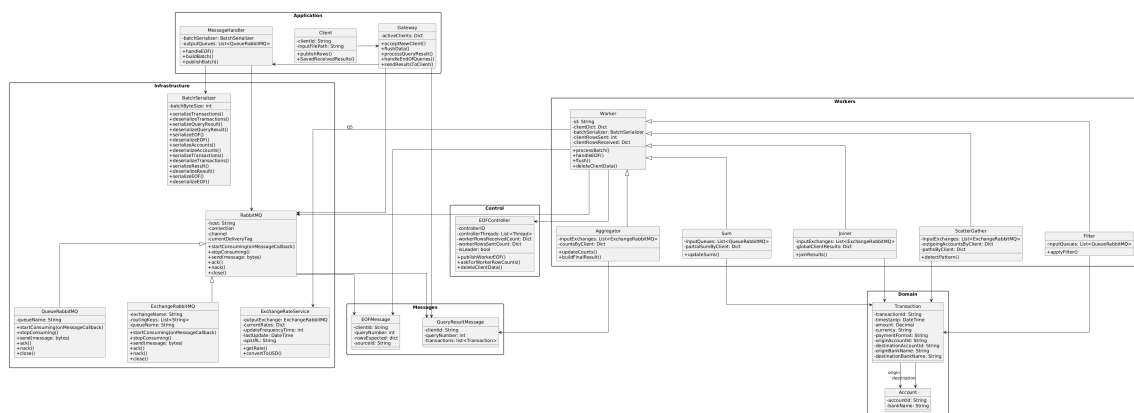


Figura 14: Diagrama de Clases

El diagrama de clases presenta las clases principales del sistema y sus relaciones. Se aclara que este diagrama nos permite visualizar las responsabilidades centrales de cada componente, pero no contiene los detalles completos de implementación.

En la capa de aplicación, el *Client* se ocupa de iniciar la comunicación con el servidor y enviarle al *Gateway* los dos archivos que contienen todas sus transacciones a analizar y sus cuentas de banco. Una vez terminado el pipeline en el lado del servidor, recibe los resultados y los almacena. Por otro lado, el *Gateway* actúa como punto único de entrada: acepta clientes, administra sus identificadores, recibe los resultados y envía las respuestas finales. El *MessageHandler* se encarga de construir batches y publicarlos en las colas correspondientes al inicio del pipeline.

La comunicación se encapsula en el paquete *RabbitMQ*. La clase *RabbitMQ* define operaciones comunes de envío, consumo, confirmación y cierre de consumo, mientras que *QueueRabbitMQ* y *ExchangeRabbitMQ* especializan esta lógica para comunicación por colas o por exchanges. La serialización de batches, queda concentrada en *BatchSerializer*.

El paquete *Workers* contiene la clase *Worker*, que reúne el comportamiento común de las etapas de procesamiento: consumo de batches, manejo de EOF, liberación de resultados acumulados y limpieza del estado de un cliente. Las clases *Filter*, *Sum*, *Joiner*, *ScatterGather* y *Aggregator* representan las operaciones lógicas necesarias para resolver las consultas. Cada worker mantiene diccionarios internos por cliente, permitiendo procesar múltiples clientes concurrentemente sin mezclar datos ni resultados.

El *EOFController* coordina el cierre de cada etapa del pipeline. A partir de los conteos de filas enviadas y recibidas, determina cuándo una etapa terminó de procesar su entrada y puede propagar el EOF o liberar sus resultados.

Finalmente, el paquete *Domain* contiene las entidades del problema: *Transaction* y *Account*. Estas modelan las transferencias bancarias y las cuentas involucradas, y son utilizadas por los distintos workers durante el procesamiento, para resolver las cinco queries.

4.7. Vista Física

En la siguiente sección figura la infraestructura de despliegue para el funcionamiento del sistema distribuido. Como mencionamos anteriormente, en el diagrama de robustez, los nodos/workers escalan horizontalmente, es decir aumentando más réplicas de los mismos. Esto se ve en el siguiente diagrama, junto con los demás nodos involucrados:

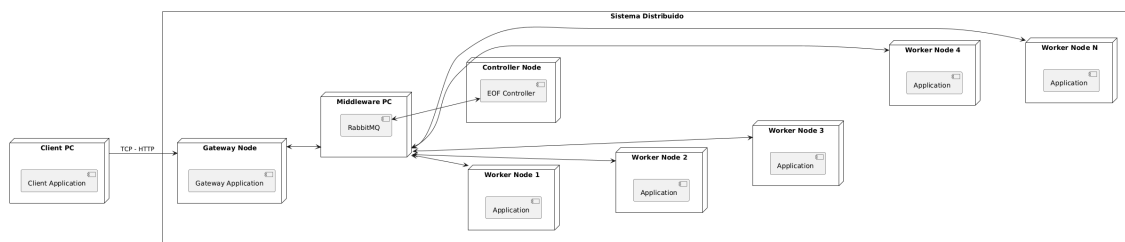


Figura 15: Diagrama de Despliegue

El diagrama de despliegue muestra la infraestructura física sobre la que corre el sistema distribuido. Se pueden identificar cinco tipos de nodos:

- **Client PC:** ejecuta la aplicación cliente, que se conecta al Gateway Node mediante TCP/IP para enviar las transacciones y recibir los resultados de las cinco consultas.
- **Gateway Node:** punto de entrada único al sistema. Ejecuta la aplicación Gateway, que recibe los datos del cliente, los distribuye al pipeline de procesamiento y recolecta los resultados para devolverlos al cliente.

- **Middleware PC:** Tiene la instancia de RabbitMQ que actua como broker de mensajería central para toda la comunicacion entre workers. Todos los nodos del sistema se conectan a este broker para publicar consumir mensajes.
- **Controller Node:** ejecuta el Controller responsable de coordinar la propagacion del EOF signal que indica fin de transmision entre etapas y gestionar el ciclo de vida de cada cliente activo.
- **Worker Nodes (1...N):** cada nodo ejecuta una instancia de la aplicacion worker, que puede corresponder a cualquiera de los workers del pipeline. El sistema escala horizontalmente agregando mas Worker Nodes, lo que permite aumentar el numero de replicas de cualquier etapa del pipeline.

5. Tareas a implementar

5.1. División entre integrantes

Tarea	Santiago Fassio	Mateo Alvarez	Cristhian Noriega
Estructura del proyecto y módulos		✓	
Docker-compose, Makefile y configuración	✓	✓	✓
Wrapper RabbitMQ y middleware	✓	✓	✓
Protocolo de mensajes y serialización			✓
Filters	✓		
Sums		✓	
Aggregators			✓
Joiners			✓
Scatter Gathers		✓	
Client Endpoint		✓	
Gateway		✓	
Exchange Rate Service			✓
Q1 Filter	✓		
Q2 SUM, Aggregator y Join			✓
Q3 SUM, JOIN y Filter	✓		
Q4 Mapper, Linker y Detector		✓	
Q5 Filter, Joiner y Aggregator			✓
EOF - Controller	✓		
Funcionalidad de escalabilidad			✓
Pruebas de integración (make test)	✓		
Graceful quit / SIGTERM		✓	

Cuadro 3: División de tareas entre integrantes